

Computer Architecture

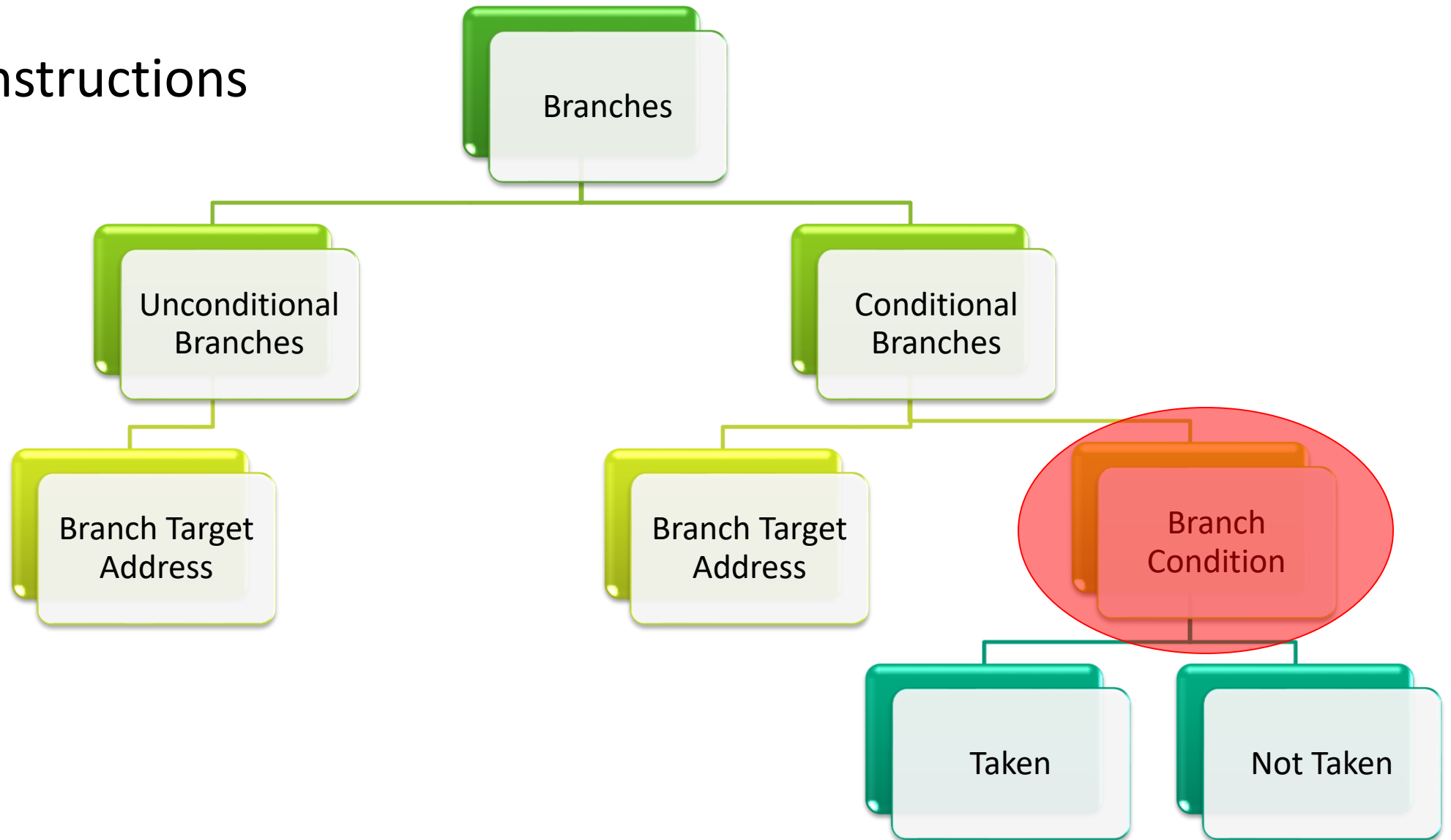
CE/CS 321/330

L2

Course Teacher: Dr.-Ing. Shehzad Hasan

Lecture: 23

Branch Instructions



Control Hazards - Branch Prediction

The idea is to guess outcome of branch's condition test (i.e. whether or not the branch will be taken).

1. Static Branch Prediction

When branch prediction is not based on branch behavior i.e. the predictor always predicts the branch in the same direction (taken/not-taken), such prediction is called static branch prediction.

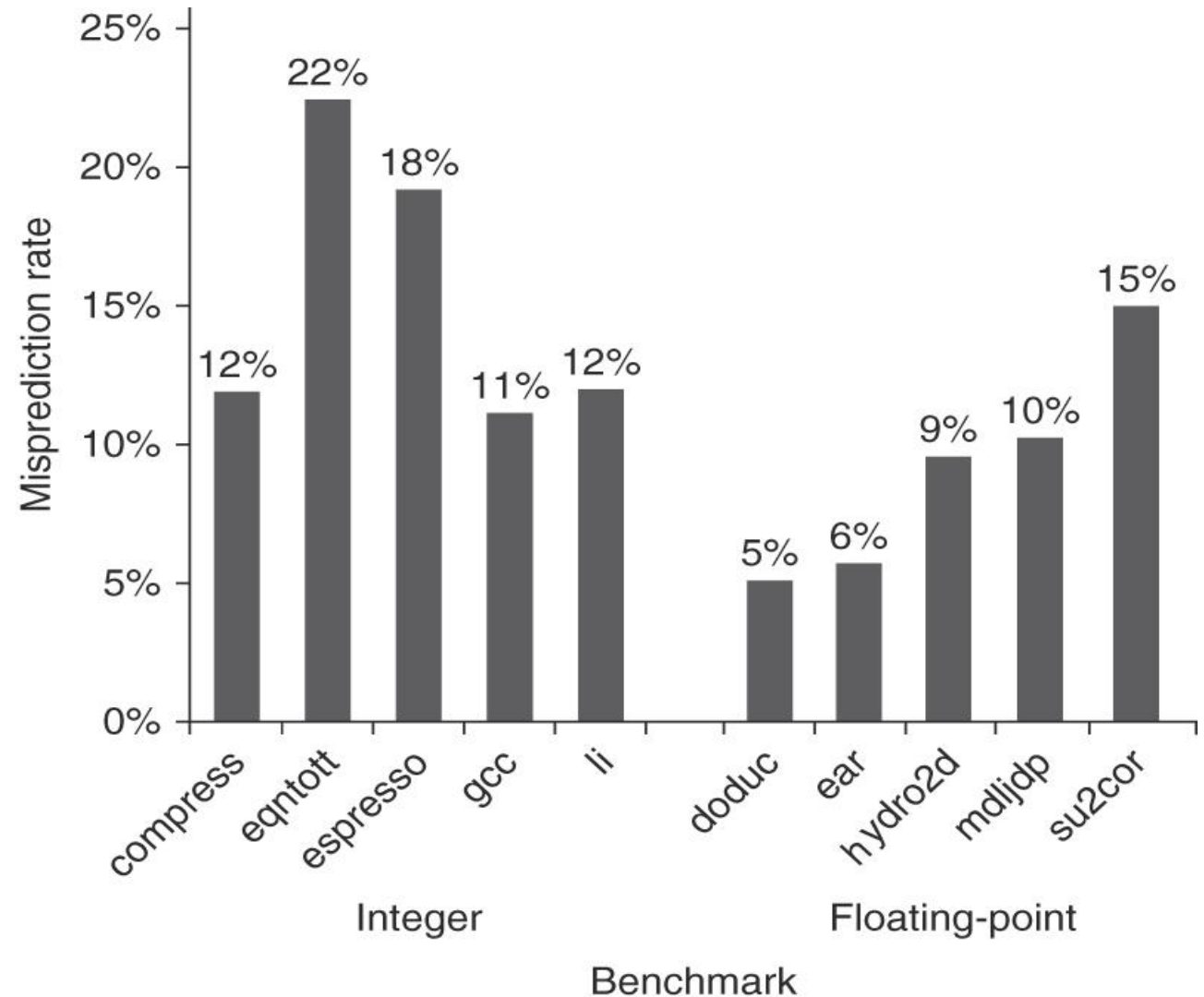
Static prediction can be done in two main ways:

- By hardware (static policy in CPU):
 - e.g., “Always predict not taken”
 - or, “Predict backward branches as taken, forward as not taken”
- By compiler:
 - Compiler reorders code and inserts delayed branch slots or NOPs according to expected branch behavior.

Control Hazards - Branch Prediction

How good is Static Branch Prediction?

- For some benchmark programs we can see a misprediction of 5% only, i.e. of all branches in the code 95% were correctly predicted by the static scheme



Heuristics for Static Branch Prediction

Heuristic	Description
Loop Branch	If the branch target is back to the head of a loop, predict taken.
Pointer	If a branch compares a pointer with NULL, or if two pointers are compared, predict in the direction that corresponds to the pointer being not NULL, or the two pointers not being equal.
Opcode	If a branch is testing that an integer is less than zero, less than or equal to zero, or equal to a constant, predict in the direction that corresponds to the test evaluating to false.
Guard	If the operand of the branch instruction is a register that gets used before being redefined in the successor block, predict that the branch goes to the successor block.
Loop Exit	If a branch occurs inside a loop, and neither of the targets is the loop head, then predict that the branch does not go to the successor that is the loop exit.
Loop Header	Predict that the successor block of a branch that is a loop header or a loop pre-header is taken.
Call	If a successor block contains a subroutine call, predict that the branch goes to that successor block.
Store	If a successor block contains a store instruction, predict that the branch does not go to that successor block.
Return	If a successor block contains a return from subroutine instruction, predict that the branch does not go to that successor block.

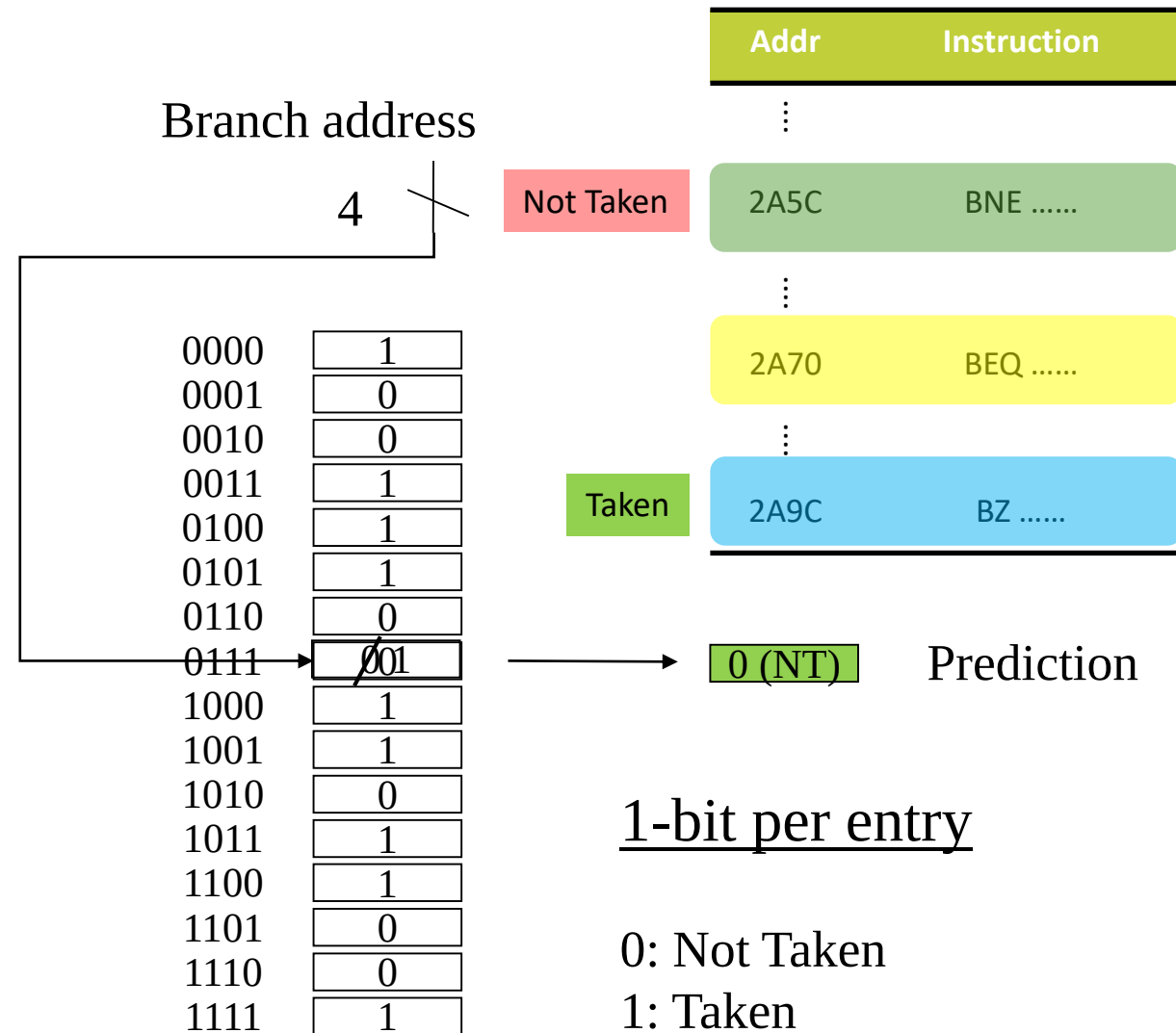
Control Hazards - Branch Prediction

Dynamic Branch Prediction

- The goal of dynamic branch prediction is to make use of run-time behavior of branch to more accurately predict what direction (taken or not-taken) a given branch will follow.
- The idea is to somehow use the branch's previous behavior to predict the current.
- The simplest dynamic branch-prediction scheme is a branch-prediction buffer or Branch History Table (BHT)

Dynamic Branch Prediction - Branch History Table

- A branch history table is a small memory indexed by the lower portion of the address of the branch instruction.
- The memory contains a bit that says whether the branch was recently taken or not.
- The prediction is just a hint, if the hint turns out to be wrong, the prediction bit is inverted and stored back.



Dynamic Branch Prediction - Branch History Table

Drawback

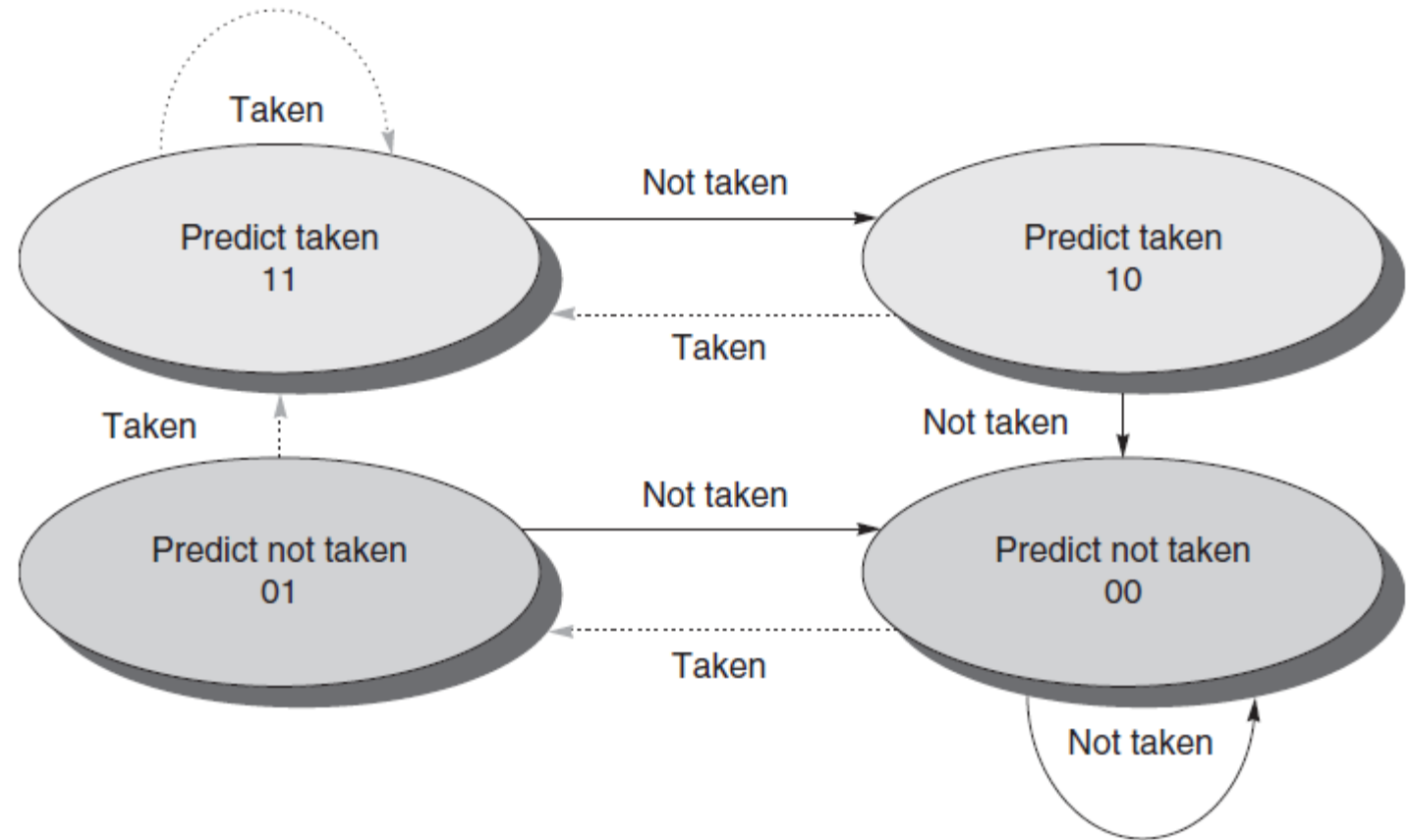
- With such a buffer, we don't know, if the prediction is correct—it may have been put there by another branch that has the same low-order address bits.

Another Drawback

- This simple 1-bit prediction scheme has a performance shortcoming:
 - Even if a branch is almost always taken, we will likely predict incorrectly twice, rather than once.
 - once upon loop exit and then again on loop entry

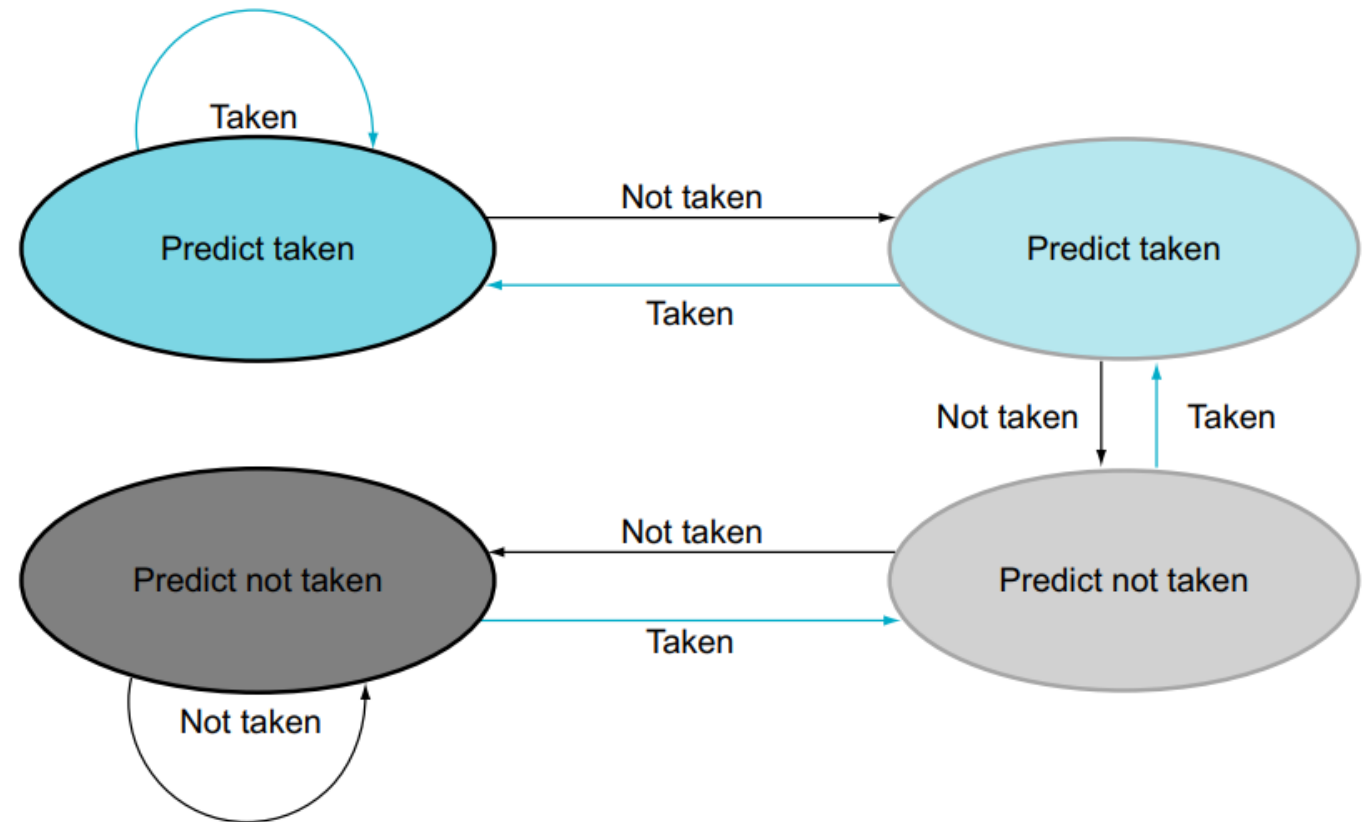
Dynamic Branch Prediction - Branch History Table

- To remedy this weakness, 2-bit prediction schemes are often used.
- In a 2-bit scheme, a prediction must miss twice before it is changed.



Dynamic Branch Prediction - Branch History Table

- Another implementation could be a 2-bit saturating counter
 - If branch is taken we add 1
 - If branch is not taken we subtract 1
 - At both ends we can't add or subtract more (saturation arithmetic)



Dynamic Branch Prediction - Branch History Table

for loop 1 (runs 1000 times)
for loop 2 (runs 100 times)
⋮
⋮
branch 2
branch 1

Misprediction rate

1-bit BHT : $2 \times 1000 / 100000 = 2\%$

2-bit BHT : $(3 + 1 \times 999) / 100000 = 1.002\%$

1-bit BHT – initial prediction: NT

1st iteration : $\frac{\text{Predicted}}{\text{Actual}} : \frac{NT}{T}, \frac{T}{T}, \dots, \frac{T}{NT}$ (2 mispredictions)

2nd iteration : $\frac{\text{Predicted}}{\text{Actual}} : \frac{NT}{T}, \frac{T}{T}, \dots, \frac{T}{NT}$ (2 mispredictions)

3rd iteration : $\frac{\text{Predicted}}{\text{Actual}} : \frac{NT}{T}, \frac{T}{T}, \dots, \frac{T}{NT}$ (2 mispredictions)

2-bit BHT – initial prediction: NT

1st iteration : $\frac{\text{Predicted}}{\text{Actual}} : \frac{NT}{T}, \frac{NT}{T}, \frac{T}{T}, \dots, \frac{T}{NT}$ (3 mispredictions)

2nd iteration : $\frac{\text{Predicted}}{\text{Actual}} : \frac{T}{T}, \frac{T}{T}, \frac{T}{T}, \dots, \frac{T}{NT}$ (1 misprediction)

3rd iteration : $\frac{\text{Predicted}}{\text{Actual}} : \frac{T}{T}, \frac{T}{T}, \frac{T}{T}, \dots, \frac{T}{NT}$ (1 misprediction)